MQCache - Redis-like cache over Microsoft Message Queuing

Version 1.0.3

Hadi Cahyadi

2026-10-12

- 1 Introduction
  - 1.1 Why MSMQ as a transport?
  - 1.2 Trade-offs (read this before adopting)
  - 1.3 Architecture
  - 1.4 Delivery semantics
- 2 MSMQ feature matrix
  - 2.1 Platform notes
- 3 Installation
  - 3.1 Requirements
  - 3.2 1. Install MSMQ
  - 3.3 2. Install the server as a service
  - 3.4 3. Try it
  - 3.5 Remote clients
  - 3.6 Upgrading
  - 3.7 Uninstall
  - 3.8 Build from source
- 4 Configuration reference
  - 4.1 MSMQ
  - 4.2 Store
  - 4.3 Security
  - 4.4 Persistence
  - 4.5 Pub/Sub
  - 4.6 Logging
  - 4.7 `CONFIG` command
  - 4.8 Exit codes of `mqcache-server`
- 5 Command reference
  - 5.1 Strings
  - 5.2 Keys and expiry
  - 5.3 Hashes
  - 5.4 Lists
  - 5.5 Sets
  - 5.6 Server
  - 5.7 Pub/Sub
  - 5.8 MSMQ administration
  - 5.9 Pipelines and atomicity

    - 5.10 Not supported
- 6 Client library (.NET)
    - 6.1 Connecting
    - 6.2 Options
    - 6.3 Typed helpers
    - 6.4 Raw commands, pipelines, per-call options
    - 6.5 Fire-and-forget with acknowledgment
    - 6.6 Pub/Sub
    - 6.7 Patterns
- 7 Command line client
- 8 Wire protocol (version 1)
    - 8.1 MSMQ message properties
    - 8.2 Request body
    - 8.3 Response body
    - 8.4 Publication body
    - 8.5 Interoperability rules for SDK authors
    - 8.6 Limits and validation
- 9 Operations guide
    - 9.1 Monitoring
    - 9.2 Persistence and backup
    - 9.3 Capacity and eviction
    - 9.4 Dead-letter, journal and poison queues
    - 9.5 Security hardening checklist
    - 9.6 Performance tuning
    - 9.7 Troubleshooting
    - 9.8 Known limitations
- 10 Development, testing and releases
    - 10.1 Repository layout
    - 10.2 Build and test
    - 10.3 Adding a command
    - 10.4 Continuous integration
    - 10.5 Releasing
    - 10.6 Building the documentation
    - 10.7 Compatibility note
- 11 FAQ
- 12 Python SDK
    - 12.1 API summary
    - 12.2 How it differs from the .NET client
    - 12.3 Testing

◦ 12.4 Writing another SDK

1 Introduction

MQCache is an in-memory cache server with a Redis-like command set whose **transport is Microsoft Message Queuing (MSMQ)** instead of TCP sockets. It runs on Windows only, because MSMQ is a Windows technology.

Component	Binary / package	Purpose
Server	`mqcache-server.exe`	Console app and Windows Service. Owns the in-memory keyspace.
CLI	`mqcache-cli.exe`	Interactive shell and one-shot runner (like `redis-cli`).
Client library	NuGet `MQCache.Client`	.NET API (`CacheClient`, `CacheSubscriber`).
Config	`mqcache.conf`	`redis.conf`-style configuration file.

1.1 Why MSMQ as a transport?

- **Durable, store-and-forward delivery.** A request survives a server restart: it waits in the queue until the server is back.
- **Transactions.** A request is received inside an MSMQ transaction; if processing fails it is redelivered.
- **Decoupling.** Clients and server do not need to be online at the same time, and the OS handles retries and routing.
- **Built-in Windows operations tooling.** Queue depth, journal, dead-letter queue and security are visible in Computer Management and Performance Monitor.

1.2 Trade-offs (read this before adopting)

MSMQ is a message queue, not a socket. Expect **higher latency and lower throughput than Redis over TCP** (typically hundreds of microseconds to a few milliseconds per round trip, dominated by MSMQ, not by the cache). Use MQCache when you want Redis-style semantics inside a Windows/MSMQ estate; use Redis when you need raw speed.

Other deliberate limits:

- Messages are limited to **4 MiB** by MSMQ, so values are limited to **3.5 MiB** (default 1 MiB).
- No blocking commands (`BLPOP`, …) and no `MULTI/WATCH`; use pipelines instead (see chapter 5).

- Single logical database, no clustering/replication. Scale by running several servers (different queue names).
- Only **private queues** and `DIRECT` format names are used, so Active Directory is not required.

1.3 Architecture

```
 client process                                          server process (service)
+-------------------------+     request message        +-------------------------------+
| CacheClient             |  (private$\mqcache queue)   | N worker threads              |
|  - reply queue          | -------------------------> |  receive in MSMQ transaction  |
|    private$\mqcache.reply|                            |  decode -> auth -> dedupe     |
|  - Send (tx or not)     |     response message       |  execute against KeyspaceStore|
|  - ReceiveByCorrelation | <------------------------- |  commit -> send reply         |
+-------------------------+  (CorrelationId = request) |  poison queue / dead-letter   |
                                                       +-------------------------------+
 subscriber queue  <----- published messages ---------- PUBLISH fan-out (or MSMQ multicast)
```

1. The client creates a private **reply queue** `mqcache.reply.<pid>.<starttime>.<rand>` and removes it on dispose (orphans of crashed processes are swept at the next start).
2. A request message carries one or more commands, the client's password, a client-generated **request id** (idempotency key) and the reply queue as `ResponseQueue`.
3. A server worker receives it inside a `MessageQueueTransaction`, executes the commands and commits, then sends the response with `CorrelationId = request message id`.
4. The client uses **selective receive** (`ReceiveByCorrelationId`), so many threads can share one reply queue.

1.4 Delivery semantics

Situation	Behaviour
Server down	Requests wait in the queue (up to their TTL, default `timeout x (retries+1)`), then expire to the dead-letter queue.
Server crashes mid-request (transactional queue)	Transaction aborts, request is redelivered.
Request redelivered or client retries after a timeout	Same request id: the server returns the cached response instead of executing again (window `dedupe-window-seconds`).
Server restarts	Keyspace is restored from the last snapshot (see chapter 9); the dedupe cache is **not** persisted.
Reply lost / client gone	Reply expires after `reply-ttl-seconds` ; no queue growth.
Poison message	After `poison-max-attempts` (or immediately when undecodable) it is moved to the poison queue and the client gets an error.

> Cache semantics, not database semantics: data between snapshots is lost on a crash. Do not use MQCache as a system of record.

2 MSMQ feature matrix

The design started from a survey of MSMQ capabilities (Microsoft Learn, the Message Queuing 4.0 feature list, and the .NET `System.Messaging` API). This chapter lists each capability, how MQCache uses it, and what is deliberately not used.

MSMQ capability	Used?	How / where	Config or API
Private queues	Yes	Request, poison, reply and subscriber queues	`queue`, `poison-queue`
Public queues (Active Directory)	No	Needs AD; the .NET Core port excludes AD integration	-
Direct format names (`OS:`, `TCP:`)	Yes	Remote servers/clients: `FormatName:DIRECT=TCP:10.0.0.5\private$\mqcache`	`endpoint tcp:10.0.0.5`
HTTP/HTTPS format names	No	Not exposed; possible by passing a custom endpoint in a fork	-
Transactional queues	Yes	Exactly-once receive, redelivery on failure	`queue-transactional`
Non-transactional queues	Yes	Lower overhead, replies, pub/sub	`queue-transactional no`
Transaction-type auto-detection	Yes	Client retries with the other send type on `TransactionUsage`	automatic
Recoverable (persistent) vs express messages	Yes	Requests persistent by default; replies/publications express	`CacheClientOptions.Recoverable`, `reply-recoverable`
Message priority	Yes	8 levels; `PING`/admin requests can jump the backlog	`CacheClientOptions.Priority`, `CallOptions.Priority`
`TimeToBeReceived`	Yes	Requests, replies and publications expire instead of piling up	client `Timeout`, `reply-ttl-seconds`
`TimeToReachQueue`	Not set	MSMQ default (follows TTBR)	-
Dead-letter queue (`UseDeadLetterQueue`)	Yes	Expired/undeliverable requests; admin via `DEADLETTER`	`CacheClientOptions.UseDeadLetter`
Transactional dead-letter queue	Yes	Same, for transactional messages	`DEADLETTER` checks both

MSMQ capability	Used?	How / where	Config or API
Journal queue (source/target)	Yes	Target journal of the request queue = audit trail; message journal flag	`queue-journal`, `UseJournal`
Journal quota	Yes	`MaximumJournalSize`	`queue-journal-max-kb`
Queue quota	Yes	`MaximumQueueSize`	`queue-max-size-kb`
Acknowledgments (`AcknowledgeType`, admin queue)	Yes	`Send(cmd, waitForReceiveAck: true)`	API
Selective receive by correlation id	Yes	Request/reply matching, shared reply queue	automatic
Cursors / enumerators (peek without removing)	Yes	`QUEUE INFO`, `POISON PEEK`, `DEADLETTER PEEK`, selective purge	admin commands
Message labels	Yes	`mqcache.req`, `mqcache.resp`, `mqcache.pub`, `mqcache.poison:*` (visible in MMC); used to touch only our messages in shared system queues	-
Poison-message handling	Yes (own)	MSMQ 4.0 retry sub-queues are WCF-driven; MQCache implements its own attempt counter and poison queue	`poison-max-attempts`, `POISON *`
Sub-queues (MSMQ 4.0)	No	Not exposed by the .NET Core port; the poison queue is a normal queue	-
Authentication (`Authenticate`, sender cert)	Optional	Queue flag; requires AD/ certificates	`queue-authenticate`
Encryption (`EncryptionRequired`)	Optional	`none/optional/body`; requires AD/certificates	`queue-encryption`
Queue ACLs	Yes	`SetPermissions` for send / full-control accounts	`queue-allow`, `queue-admin`
Multicast (PGM)	Optional	Server publishes selected channels to a multicast group; clients can join	`multicast-channel`, `SubscribeMulticast`
Tracing / report queue	Flag only	`UseTracing` can be set per client; report queue must be configured in MSMQ	`CacheClientOptions.UseTracing`

MSMQ capability	Used?	How / where	Config or API
Queue creation with properties, labels, categories	Yes	Created and (re)configured at start	-
Orphan-queue hygiene	Yes	Reply queues named after pid+start time, swept by later clients	`SweepOrphanQueues`
Distribution lists, queue aliases, MQIS/AD lookup	No	Require Active Directory	-
Message routing servers	N/A	Provided by MSMQ itself between machines	-
Per-application dead-letter queue (MSMQ 4.0, `PROPID_M_DEADLETTER_QUEUE`)	No	Not exposed by `System.Messaging`	-
Transactional remote receive (MSMQ 4.0)	No	Clients receive replies from a local queue	-

2.1 Platform notes

- The .NET 8 API is the community port `Experimental.System.Messaging` 1.2.0 (unofficial, **Ms-RL** licensed). MQCache itself is MIT; the dependency is a package reference only.
- Microsoft documents MSMQ as a feature that must be installed (Windows feature `MSMQ-Server`) and only recommends it for existing estates; evaluate this against your own support policy.

3 Installation

3.1 Requirements

- Windows 10/11 or Windows Server 2016+ (x64 or ARM64).
- The **Microsoft Message Queue (MSMQ)** feature and the *Message Queuing* service running.
- For building: .NET SDK 8.0. Released binaries are self-contained.

3.2 1. Install MSMQ

```
# Windows client (10/11)
Enable-WindowsOptionalFeature -Online -FeatureName MSMQ-Server -All
# Windows Server
Install-WindowsFeature MSMQ-Server
Get-Service MSMQ   # must be Running
```

Check that the queue manager works: `Get-MsmqQueue` (module `MSMQ`) or Computer Management > Services and Applications > Message Queuing.

3.3 2. Install the server as a service

Download the zip for your platform from the GitHub release, extract it, and from an elevated PowerShell:

```
.\install-service.ps1 -EnableMsmq -OpenFirewall
```

The script copies binaries to `%ProgramFiles%\MQCache`, writes `%ProgramData%\MQCache\mqcache.conf` (never overwritten on upgrade), registers the service `MQCache` (delayed auto-start, depends on MSMQ, restart on failure), and optionally opens the MSMQ firewall ports for remote clients. Default account: `NT AUTHORITY\NetworkService`.

Manual alternative:

```
sc.exe create MQCache binPath= "\"C:\mqcache\mqcache-server.exe\" \"C:\mqcache\mqcache.conf\"" start=
delayed-auto depend= MSMQ
```

Run in a console for testing: `mqcache-server.exe conf\mqcache.conf` (Ctrl+C stops it cleanly and saves a snapshot).

Validate a config without starting: `mqcache-server.exe --test-config conf\mqcache.conf`.

3.4 3. Try it

```
mqcache-cli.exe PING
mqcache-cli.exe SET greeting hello EX 60
mqcache-cli.exe GET greeting
```

3.5 Remote clients

- Server and client machines both need MSMQ. Use `-e tcp:<server-ip>` (recommended) or `-e os:<server-name>`.
- Firewall (inbound on the **server**, and on the **client** for replies): TCP 1801, TCP 135/2103/2105 (RPC, remote read), UDP 3527. Multicast additionally needs the chosen group/port.
- The server answers to the client's **reply queue**, so the client must be reachable: set `ReplyHost` (name or IP) when the machine name does not resolve from the server; for `tcp:` use `ReplyScheme = "tcp"` with an IP.
- Workgroup (non-domain) setups: MSMQ then runs without AD; keep `queue-authenticate no` and rely on `requirepass` plus network controls.

3.6 Upgrading

Stop the service, replace the executables, start the service. The snapshot format is versioned; a snapshot written by an older version is read by newer ones. Changing `queue-transactional` requires deleting the queue after it is drained.

3.7 Uninstall

`.\uninstall-service.ps1 [-RemoveData]`. Queues `private$\mqcache*` are kept; delete them in Computer Management if wanted.

3.8 Build from source

```
git clone https://github.com/cumulus13/MQCache.git
cd MQCache
dotnet build -c Release
dotnet test tests/MQCache.Core.Tests
dotnet run --project src/MQCache.Server -- conf/mqcache.conf
```

4 Configuration reference

`mqcache.conf` is `redis.conf` style: `directive value [value...]`, `#` starts a comment, use double quotes for values containing spaces or `#`. Unknown directives are an **error** (typos are caught at startup). Any directive may be overridden on the command line: `mqcache-server mqcache.conf --workers 8 --maxmemory 512mb`.

4.1 MSMQ

Directive	Default	Description
`queue`	`private$\mqcache`	Request queue (private, local). Clients address it as `<host>/<name>`.
`queue-transactional`	`yes`	Transactional queue (exactly-once receive, redelivery). Cannot be changed on an existing queue.
`queue-journal`	`no`	Keep a copy of consumed requests in the queue journal (audit).
`queue-journal-max-kb`	`0`	`MaximumJournalSize` (0 = MSMQ default).
`queue-max-size-kb`	`0`	`MaximumQueueSize` (0 = MSMQ default).
`queue-authenticate`	`no`	Require authenticated messages (needs AD/certificates).
`queue-encryption`	`none`	`none`, `optional`, `body` (needs AD/certificates).
`queue-allow`	-	Account with SEND rights (repeatable).
`queue-admin`	-	Account with full control (repeatable).
`workers`	`4`	Parallel receive loops (1..64). `1` gives strict global request ordering.
`receive-timeout-ms`	`1000`	Receive poll timeout; also the shutdown latency.
`reply-ttl-seconds`	`60`	`TimeToBeReceived` of replies/publications.
`reply-recoverable`	`no`	Persist replies to disk.

Directive	Default	Description
`poison-queue`	`private$\mqcache.poison`	Where failing requests are parked.
`poison-max-attempts`	`3`	Attempts before a failing request is parked.
`dedupe-window-seconds`	`60`	Idempotency window (0 = off).
`dedupe-max-entries`	`100000`	Bound of the idempotency cache.

4.2 Store

Directive	Default	Description
`maxmemory`	`0`	Memory limit for data (accounted by the store, not process RSS). `64kb`, `512mb`, `2gb`.
`maxmemory-policy`	`noeviction`	`noeviction`, `allkeys-lru`, `volatile-lru`, `volatile-ttl`.
`max-value-bytes`	`1048576`	Largest argument of a write (max 3670016 because of the 4 MiB MSMQ limit).
`max-batch-commands`	`1000`	Commands per pipeline.
`keys-max-results`	`10000`	Cap of `KEYS`.
`active-expire-interval-ms`	`100`	Expired-key sweep period.

With `noeviction`, write commands that add memory return `-OOM` once the limit is reached; reads and deletes keep working. With an eviction policy, least-recently-used keys are evicted until usage fits.

4.3 Security

Directive	Description
`requirepass`	Password required in every request (compared in constant time). It travels inside the message: combine with queue ACLs, `queue-encryption` (domain) or a trusted network.

4.4 Persistence

Directive	Default	Description
`dir`	`./data`	Snapshot directory.

Directive	Default	Description
`dbfilename`	`dump.mqdb`	Snapshot file name.
`save <seconds> <changes>`	`900 1`, `300 100`, `60 10000`	Repeatable. `save ""` disables snapshots.
`save-on-shutdown`	`yes`	Final snapshot on clean stop.

4.5 Pub/Sub

Directive	Default	Description
`pubsub-subscription-ttl-seconds`	`120`	Subscriber lease; clients renew every 30 s.
`multicast-channel <channel> <ip:port>`	-	Also publish to an MSMQ multicast group (repeatable).

4.6 Logging

Directive	Default	Description
`loglevel`	`info`	`debug`, `info`, `warn`, `error`.
`logfile`	-	Daily rolling file `name-yyyyMMdd.log`. As a service the default is `.\logs\mqcache-server.log`.

4.7 `CONFIG` command

`CONFIG GET <pattern>` returns effective values (`requirepass` is masked). `CONFIG SET maxmemory|maxmemory-policy <value>` changes those two at runtime.

4.8 Exit codes of `mqcache-server`

`0` clean stop, `1` fatal error (for example MSMQ unavailable), `2` configuration error.

5 Command reference

Command names are case-insensitive. Keys, fields and set members are UTF-8 text; values are binary-safe. Replies follow Redis conventions (status, error, integer, bulk, nil, array). Errors carry Redis-style prefixes (`ERR`, `WRONGTYPE`, `OOM`, `NOAUTH`, `WRONGPASS`).

5.1 Strings

Command	Description
`GET key`	Value or nil.
`SET key value [EX s] [PX ms] [NX] [XX] [KEEPTTL]`	`OK`, or nil when the NX/XX condition fails.
`SETNX key value`	`1` if set.
`SETEX key seconds value` / `PSETEX key ms value`	Set with TTL.
`GETSET key value`	Set and return the old value.
`APPEND key value`	New length.
`STRLEN key`	Length.
`INCR` / `DECR key`, `INCRBY` / `DECRBY key n`	64-bit integer arithmetic; overflow is an error.
`MGET key...` / `MSET key value...`	Multiple get/set.

5.2 Keys and expiry

Command	Description
`DEL key...` / `EXISTS key...`	Counts.
`TYPE key`	`string`, `hash`, `list`, `set`, `none`.
`KEYS pattern`	Glob (`* ? [abc] [a-z] [^x] \`). Capped by `keys-max-results`. O(N).
`RENAME src dst`	Keeps TTL; overwrites `dst`.
`EXPIRE key s` / `PEXPIRE key ms`	Non-positive TTL deletes the key.
`TTL key` / `PTTL key`	`-2` missing, `-1` no TTL.
`PERSIST key`	Remove TTL.
`DBSIZE`, `FLUSHALL`, `FLUSHDB`	Single database; both flush commands are equivalent.

5.3 Hashes

`HSET key field value [field value...]` (returns new fields), `HMSET`, `HGET`, `HMGET`, `HDEL`, `HGETALL`, `HKEYS`, `HVALS`, `HLEN`, `HEXISTS`, `HINCRBY key field n`.

5.4 Lists

`LPUSH`, `RPUSH key value...`, `LPOP`, `RPOP` (single element), `LLEN`, `LRANGE key start stop` (negative indexes allowed), `LINDEX key index`. Index access is O(n).

5.5 Sets

`SADD`, `SREM`, `SMEMBERS`, `SISMEMBER`, `SCARD`. Empty hashes/lists/sets are deleted automatically.

5.6 Server

Command	Description
`PING [msg]`, `ECHO msg`, `TIME`	Connectivity and clock.
`INFO`	Server, stats, memory, persistence, keyspace and `# Msmq` sections.
`SAVE` / `BGSAVE` / `LASTSAVE`	Snapshot now / in background / last save (unix seconds).
`CONFIG GET pattern`, `CONFIG SET maxmemory|maxmemory-policy v`	See chapter 4.

5.7 Pub/Sub

Command	Description
`PUBLISH channel message`	Returns the number of subscriber queues the message was sent to.
`SUBSCRIBE` / `PSUBSCRIBE ch...`	Register the request's reply queue. Normally used through `CacheSubscriber`, which owns a dedicated queue and renews the lease.
`UNSUBSCRIBE` / `PUNSUBSCRIBE [ch...]`	Remove subscriptions (all when no names).
`PUBSUB CHANNELS [pattern]`, `PUBSUB NUMSUB ch...`	Introspection.

Delivery is **at-most-once** (express messages with a TTL). A subscriber that cannot be reached three times in a row is dropped.

5.8 MSMQ administration

Command	Description
`QUEUE INFO`	Depth of request/poison/journal/dead-letter queues, subscriptions.
`DEADLETTER COUNT` / `PEEK [n]` / `PURGE`	System dead-letter queues; **only messages labelled `mqcache.*` are counted, shown or removed**.
`JOURNAL COUNT` / `PURGE`	Request-queue journal (requires `queue-journal yes`).
`POISON COUNT` / `PEEK [n]` / `PURGE` / `REQUEUE [n]`	Park management; `REQUEUE` moves messages back atomically.

5.9 Pipelines and atomicity

Several commands travel in **one MSMQ message** (`CacheClient.Pipeline`). With `atomic: true` (default) the server runs them while holding the store lock, so no other client's command interleaves; there is no rollback (like Redis `MULTI`, an error in one command does not undo the others). The whole pipeline is idempotent under retry because it shares one request id.

5.10 Not supported

Blocking commands, `WATCH/MULTI/EXEC`, `SCAN`, sorted sets, streams, Lua, multiple databases, replication/cluster, `AUTH` as a command (the password is part of each request).

6 Client library (.NET)

```
dotnet add package MQCache.Client
```

Target: `net8.0-windows`. The client is **thread-safe**; share one `CacheClient` per process. Each client owns a private reply queue (removed on `Dispose`).

6.1 Connecting

```
using MQCache.Client;

using var cache = CacheClient.Connect("tcp:10.0.0.5");                       // endpoint
using var cache2 = CacheClient.Connect("endpoint=tcp:10.0.0.5;password=s3;timeout=3000;retries=1");
using var local = new CacheClient(new CacheClientOptions { QueueName = "mqcache", Password = "s3" });
```

Endpoint grammar: `.` (local), `host`, `os:host`, `tcp:ip`, optionally `/queue` (`tcp:10.0.0.5/orders-cache`).

6.2 Options

Option	Default	Meaning
`Endpoint`, `QueueName`	`.`, `mqcache`	Where the server queue lives.
`Transactional`	`true`	Initial guess; auto-corrected if the server queue is the other kind.
`Password`	-	Sent with each request.
`Timeout`	5 s	Per attempt.
`Retries`	2	Re-sends with the **same request id** after a timeout (safe for `INCR` etc.).
`Priority`	Normal	MSMQ priority of requests.
`Recoverable`	`true`	Persistent request messages.
`UseJournal`, `UseDeadLetter`, `UseTracing`	`false`, `true`, `false`	MSMQ message flags.
`ReplyHost`, `ReplyScheme`	machine name, `os`	How the server addresses your reply queue.
`SweepOrphanQueues`	`true`	Delete reply/subscriber queues of dead processes at start.

6.3 Typed helpers

```
        cache.Set("user:1", "alice", ttl: TimeSpan.FromMinutes(5));
        cache.Set("lock:x", "me", TimeSpan.FromSeconds(10), SetWhen.NotExists);   // SET NX PX
        string? v = cache.GetString("user:1");
        long n = cache.Increment("hits");
        cache.HashSet("session:9", "ip", "10.0.0.1");
        var all = cache.HashGetAll("session:9");
        cache.ListPush("jobs", "a"); string? job = cache.ListPop("jobs");
        cache.SetAdd("tags", "x", "y");
        TimeSpan rtt = cache.Ping();
```

Error replies (`WRONGTYPE` , `OOM` , `NOAUTH` …) throw `CacheServerException` ; timeouts throw `CacheTimeoutException` ; MSMQ problems throw `CacheException` .

6.4 Raw commands, pipelines, per-call options

```
        RespValue r = cache.Execute("SET", "k", "v", "EX",
"10");           // does not throw on error replies; check r.IsError

        var results = cache.Pipeline(new[]
        {
            Command.Of("SET", "a", "1"),
            Command.Of("INCR", "a"),
            Command.Of("GET", "a"),
        });                                                          // one MSMQ round trip, atomic

        cache.Execute(Command.Of("PING"), new CallOptions { Priority = MessagePriority.Highest, Timeout =
TimeSpan.FromSeconds(1) });
```

6.5 Fire-and-forget with acknowledgment

```
        cache.Send(Command.Of("SET", "k", "v"));                              // no reply expected
        cache.Send(Command.Of("SET", "k", "v"), waitForReceiveAck: true);       // MSMQ confirms the server
consumed it
```

6.6 Pub/Sub

```
        using var sub = cache.Subscribe("news");          // or cache.PSubscribe("news.*")
        sub.MessageReceived += m => Console.WriteLine($"{m.Channel}: {m.PayloadText}");
        // or pull-style:
        foreach (var m in sub.ReadAll(token)) { /* ... */ }

        cache.Publish("news", "hello");                   // returns number of subscriber queues reached
```

A subscriber owns its queue and renews its lease every `SubscriptionRenewInterval` (30 s); disposing it unsubscribes. Messages are at-most-once. For MSMQ multicast fan-out configure `multicast-channel news 234.1.1.1:8001` on the server and

`cache.SubscribeMulticast("234.1.1.1:8001")` on clients (requires the MSMQ multicast feature).

6.7 Patterns

- **Cache-aside**: `GetString` then `Set(..., ttl)`.
- **Distributed lock**: `Set(key, token, ttl, SetWhen.NotExists)`; release with `Delete` after checking the token.
- **Rate limit**: `Pipeline(INCR key, EXPIRE key 60)`.
- **High-throughput writes**: batch into pipelines (one message per batch) and keep `Recoverable = false` if persistence in MSMQ is not needed.

7 Command line client

```
mqcache-cli [options] [command [args...]]
  -e, --endpoint <spec>   . | host | os:host | tcp:10.0.0.5     (default ".")
  -q, --queue <name>      request queue name                    (default "mqcache")
  -a, --auth <password>
  -t, --timeout <ms>      per attempt (default 5000)    --retries <n> (default 2)
      --priority <p>      lowest .. highest             --no-tx   server queue is non-transactional
      --raw               raw output (default when stdout is redirected)
```

Interactive shell:

```
local/mqcache> SET greeting "hello world" EX 60
OK
local/mqcache> GET greeting
"hello world"
local/mqcache> HGETALL session:9
1) "ip"
2) "10.0.0.1"
local/mqcache> SUBSCRIBE news          (Ctrl+C returns to the prompt)
local/mqcache> BENCHMARK 5000 8        (example output; numbers depend entirely on your hardware and MSMQ
5000 SET+GET pairs, 8 threads, 3.1s => 3225 ops/s, errors=0
```

One-shot (scripting): `mqcache-cli -e tcp:10.0.0.5 -a secret GET greeting`. Exit code is `0` on success, `1` on error replies or failures.

Quoting: both `"double"` (with `\n \r \t \"` escapes) and `'single'` quotes are accepted.

8 Wire protocol (version 1)

Useful for writing clients in other languages (C++ via the MSMQ COM/C API, PowerShell, etc.). All integers are **little-endian**; `bytes` = `int32 length` + data; strings are UTF-8 `bytes`.

8.1 MSMQ message properties

Property	Request	Response	Publication
`Label`	`mqcache.req`	`mqcache.resp`	`mqcache.pub`
`AppSpecific`	`1`	`1`	`1`
`ResponseQueue`	client reply queue (`DIRECT=` format name)	-	-
`CorrelationId`	-	id of the request message	-
`Priority`	client choice	copied from request	normal
`Recoverable`	client choice (default true)	`reply-recoverable`	false
`TimeToBeReceived`	`timeout x (retries+1)`	`reply-ttl-seconds`	`reply-ttl-seconds`
`AdministrationQueue` + `AcknowledgeType`	optional (`FullReceive`)	-	-
`BodyType`	`0x2011` (VT_ARRAY|VT_UI1)	same	same
Body	raw bytes (`BodyStream`, no formatter)	same	same

`BodyType = 0x2011` is what lets COM and C-API clients (Python, Go, Rust…) read the body as a plain byte array. No .NET serializer is involved (`BinaryFormatter` is not used), so any MSMQ client can interoperate.

8.2 Request body

```
byte    version = 1
byte    flags            bit0 = atomic, bit1 = no-reply
byte[16] requestId       GUID bytes (Guid.ToByteArray); idempotency key reused on retries
bytes   auth             password, empty if none
int32   commandCount     1..100000
repeat commandCount:
  int32 argc             name + arguments (1..65536)
  bytes name
  bytes arg  x (argc-1)
```

8.3 Response body

```
byte version = 1
byte[16] requestId
int32 count
repeat count: value
value := byte type + payload
  1 Status  bytes(text)          2 Error  bytes(text)
  3 Integer int64                4 Bulk   bytes
  5 Null    (no payload)         6 Array  int32 n, n x value
```

The response contains exactly one value per command. If the server rejects a whole message (undecodable, poison) it answers with a single `Error` and an all-zero request id.

8.4 Publication body

```
byte version = 1 ; byte kind (1 = channel, 2 = pattern) ; bytes pattern ; bytes channel ; bytes payload
```

8.5 Interoperability rules for SDK authors

- Reuse the **same request id** across retries (idempotency).
- Use your own private queue name prefix; `mqcache.reply.` and `mqcache.sub.` are reserved for the .NET client, whose sweeper deletes queues of those prefixes whose embedded process id/start time is dead.
- Conformance: reproduce `sdk/testdata/golden.json` (generated by `sdk/tools/GoldenVectors`) byte for byte. See chapter 12.

8.6 Limits and validation

Every length is checked against the remaining bytes, nesting is limited to 16 levels, and trailing bytes are rejected. Malformed requests go to the poison queue; they never crash a worker.

9 Operations guide

9.1 Monitoring

- `INFO` (via `mqcache-cli INFO`): keyspace hits/misses, evictions, memory, last save, and the `# Msmq` section (`requests_received`, `requests_redelivered`, `requests_poisoned`, `dedupe_hits`, `reply_failures`, `pubsub_subscriptions`).
- `QUEUE INFO`: `request_queue_depth` is the backlog; a growing value means the server is down or overloaded. Alert on it.
- Windows Performance Monitor: *MSMQ Queue* and *MSMQ Service* counters; Computer Management shows messages per queue, journal and dead-letter queues.
- Logs: `logs\mqcache-server-yyyyMMdd.log` (service) or console. Use `loglevel debug` temporarily.

9.2 Persistence and backup

Data lives in memory. A snapshot (`dump.mqdb`) is written atomically (`.tmp` then replace, previous file kept as `.bak`) and carries a SHA-256 trailer. Snapshots are taken by `save` rules, `SAVE` / `BGSAVE`, and on clean shutdown. On start the file is loaded; if it is corrupt it is renamed `dump.mqdb.corrupt-<time>` and the server starts empty (logged as an error).

Backup = copy `dump.mqdb` (it is always a complete, consistent file). Restore = stop the service, place the file, start.

MSMQ itself is durable: persistent (`Recoverable`) requests that arrived but were not yet processed survive a restart of MSMQ or the machine. Applied state between snapshots does not.

9.3 Capacity and eviction

Set `maxmemory` below the machine's free RAM (accounting is approximate: key and value bytes plus per-item overhead; allow ~1.5x headroom). Choose `allkeys-lru` for a pure cache, `volatile-lru` / `volatile-ttl` when only keys with TTL may be dropped, `noeviction` when you prefer errors to data loss.

Also bound MSMQ: `queue-max-size-kb` limits the request queue so a dead server cannot fill the disk; remember the machine-wide MSMQ quota (default 1 GB of persistent messages) applies to all queues, journal and dead-letter included.

9.4 Dead-letter, journal and poison queues

- **Dead-letter**: requests that expired before being consumed (for example the server was down for longer than the client's TTL). `DEADLETTER COUNT/PEEK/PURGE` only

touch `mqcache.*` messages, because the dead-letter queue is shared with other applications.
- **Journal**: with `queue-journal yes`, every consumed request is copied to `private$\mqcache\Journal$`. It is an audit trail and grows until `queue-journal-max-kb` is reached; purge regularly (`JOURNAL PURGE`).
- **Poison**: undecodable messages and requests that fail `poison-max-attempts` times. Inspect with `POISON PEEK`, fix the cause, then `POISON REQUEUE` or `POISON PURGE`.

9.5 Security hardening checklist

1. Set `requirepass` (long random value). Remember it is carried in the message.
2. Restrict queue access with `queue-allow` / `queue-admin` (domain groups) and firewall MSMQ ports to known clients.
3. In a domain: `queue-authenticate yes` and `queue-encryption body`.
4. Run the service under a low-privilege account (default NetworkService); grant it write access only to `dir` and the log directory.
5. Keep `max-value-bytes` and `max-batch-commands` at the smallest values you need.
6. Treat the cache as untrusted-input facing: keys/values are data, never executed; all inputs are length-checked.
7. Do not expose MSMQ to the internet. Use a VPN or private network.

See also `SECURITY.md`.

9.6 Performance tuning

- Latency is dominated by MSMQ. Non-transactional queue + `Recoverable = false` gives the lowest overhead; transactional + recoverable the highest durability.
- Batch with pipelines: one message for many commands.
- `workers`: 2-8 is enough; the store lock serializes commands, extra workers only overlap MSMQ I/O.
- Keep reply queues local to the client (default) and avoid DNS lookups: use `tcp:<ip>`.
- Large values cost more than many small ones; keep values small.

9.7 Troubleshooting

Symptom	Cause / fix
`MSMQ is not available (ServiceNotAvailable)`	Install the MSMQ feature and start the *Message Queuing* service.
Client times out, server log is quiet	Wrong queue name/endpoint; message sits in an *outgoing queue* (look in Computer Management > Outgoing Queues) because the remote host is unreachable or the firewall blocks TCP 1801.

Symptom	Cause / fix
Requests arrive but replies never do	Server cannot reach the client's reply queue: set `ReplyHost`, check name resolution/firewall, or use `tcp:` + IP. Check `reply_failures` in `INFO`.
`Queue ... exists with transactional=...` at start	Config does not match the existing queue. Drain and delete the queue, or change the config back.
`Access is denied` creating queues	The service account lacks rights; run once elevated, or grant the account permissions in MSMQ.
`-OOM command not allowed`	`maxmemory` reached with `noeviction`; raise the limit or choose an eviction policy.
`-NOAUTH` / `-WRONGPASS`	Client password missing/incorrect.
Orphan `mqcache.reply.*` queues	Crashed clients; the next client start removes them (local machine only).
Snapshot `...corrupt-...` appears	Disk or shutdown problem; restore from backup.

9.8 Known limitations

- Subscriptions are leases; a subscriber that is offline longer than the lease misses messages (pub/sub is at-most-once).
- The dedupe cache is in memory only; after a server restart a retried request may execute again.
- `LRANGE` / `LINDEX` are O(n); `KEYS` is O(total keys) and holds the store lock while scanning.
- Memory accounting is an estimate.

10 Development, testing and releases

10.1 Repository layout

```
src/MQCache.Core       platform-independent: store, commands, codec, config, snapshots   (net8.0)
src/MQCache.Messaging  MSMQ plumbing: endpoints, queue admin, message factory          (net8.0-windows)
src/MQCache.Server     mqcache-server: workers, router, hosting/service                (net8.0-windows)
src/MQCache.Client     MQCache.Client NuGet package                                    (net8.0-windows)
src/MQCache.Cli        mqcache-cli                                                        (net8.0-windows)
tests/MQCache.Core.Tests          unit tests (run anywhere)
tests/MQCache.IntegrationTests    real MSMQ (auto-skipped when MSMQ is missing)
conf/mqcache.conf  scripts/  docs/  .github/workflows/
```

All MSMQ-specific code is isolated in `Messaging` / `Server` / `Client`; the cache semantics live in `Core` and are unit-tested without MSMQ.

10.2 Build and test

```
dotnet build -c Release
dotnet test tests/MQCache.Core.Tests            # fast, no MSMQ needed
dotnet test tests/MQCache.IntegrationTests      # needs the MSMQ feature + service
```

10.3 Adding a command

1. Add the operation to `KeyspaceStore` (keep it atomic under the store lock; update `_dirty` and size accounting).
2. Register it in `CommandProcessor.Register()` with its arity and whether it can **add memory** (`write = true`).
3. Add unit tests in `CoreTests.cs`; document it in chapter 5.

MSMQ-aware commands (anything needing queues or the sender's reply queue) go in `ServerCommandRouter`.

10.4 Continuous integration

`.github/workflows/ci.yml` (push/PR): restore, format check (informational), build, Core unit tests with coverage, integration tests on a Windows runner with MSMQ enabled (best effort, non-blocking), NuGet pack, single-file publish smoke test, documentation build.

10.5 Releasing

1. Update `CHANGELOG.md` with a `## [x.y.z] - date` section.
2. Tag and push: `git tag v1.0.0 && git push origin v1.0.0`.

3. `release.yml` verifies the tests, builds `win-x64` and `win-arm64` single-file self-contained zips (server, CLI, config, install scripts), packs `MQCache.Client`, builds the documentation in md/html/docx/pdf/epub, writes `SHA256SUMS.txt`, creates the GitHub Release (notes from the changelog) and pushes to NuGet when the `NUGET_API_KEY` secret exists.

10.6 Building the documentation

```
choco install pandoc
pwsh docs/build-docs.ps1                          # md, html, docx, pdf, epub into dist/docs
pwsh docs/build-docs.ps1 -Formats md,docx -Out out
```

Chapters are `docs/NN-*.md`; they are concatenated in order. PDF uses xelatex, else wkhtmltopdf, else headless Edge/Chrome. Single chapters can be exported with plain pandoc: `pandoc docs/05-commands.md -o commands.docx`.

10.7 Compatibility note

The MSMQ API comes from `Experimental.System.Messaging` (community port, Ms-RL). If Microsoft ships an official .NET MSMQ client, only `MQCache.Messaging`, `Server` and `Client` need their `using` lines changed.

11 FAQ

Is this compatible with Redis clients? No. It mimics the command set and reply types, not the RESP/TCP protocol. Use `MQCache.Client` or implement chapter 8.

Why not just use Redis? Use Redis if you can. MQCache exists for estates standardised on MSMQ where opening TCP services is undesirable, or where store-and-forward behaviour for cache writes is valuable.

Can several servers share one queue? Not for one dataset (each holds its own memory). Run separate servers with different `queue` names and shard in the client.

Does it work on Linux? No. MSMQ is Windows-only. `MQCache.Core` builds anywhere and is reusable.

What happens if the client sends while the server is down? The message waits (persistent) until its TTL expires, then moves to the dead-letter queue; the client sees a timeout after its retries.

How do I run two environments side by side? Use different `queue` names, `dir`, and `workers`; clients choose with `QueueName`.

12 Python SDK

Package `mqcache` (`sdk/python`), Python 3.9+, Windows (it drives MSMQ through its COM automation objects with `pywin32`).

```
pip install mqcache
```

```
from mqcache import CacheClient, Priority

with CacheClient("tcp:10.0.0.5", password="secret", decode_responses=True) as c:
    c.set("user:1", "alice", ex=300)
    c.incr("hits")
    c.hset("session:9", mapping={"ip": "10.0.0.1"})
    c.execute_command("PING", priority=Priority.HIGHEST)
    results = c.pipeline([("SET", "a", 1), ("INCR", "a"), ("GET", "a")])     # one message, atomic

    with c.subscriber("news", patterns=["alerts.*"]) as sub:
        c.publish("news", "hello")
        for msg in sub.listen():          # or sub.get_message(timeout=5)
            print(msg.channel, msg.text)
```

12.1 API summary

Area	Methods
Core	`execute_command(*args)`, `pipeline(cmds, atomic=True, raise_on_error=False)`, `send_no_reply(*args)`
Strings/keys	`get set(ex, px, nx, xx, keepttl) delete exists incr decr append expire pexpire ttl pttl persist type keys mget mset rename dbsize flushall save`
Hashes	`hset(key, field, value, mapping) hget hdel hlen hexists hincrby hgetall`
Lists/sets	`lpush rpush lpop rpop llen lrange lindex`, `sadd srem smembers sismember scard`
Server	`ping info config_get publish`
Pub/Sub	`subscriber(*channels, patterns=...)` with `subscribe psubscribe unsubscribe get_message listen close`
Options	`endpoint queue password timeout retries priority recoverable transactional use_journal use_dead_letter reply_host reply_scheme decode_responses subscription_renew transport`

Errors: `ServerError` (`.code` is `ERR`, `WRONGTYPE`, `OOM`, `NOAUTH`, …), `CacheTimeoutError`, `TransportError`, `MsmqUnavailableError`, `WireFormatError`; all derive from `MQCacheError`.

12.2 How it differs from the .NET client

- No selective receive in the COM API, so replies for all threads are read by one waiter at a time and handed to their owners (`_ReplyDemux`).
- Queue names use `mqcache.pyreply.` / `mqcache.pysub.` so the .NET orphan sweeper (which owns `mqcache.reply.` / `mqcache.sub.`) never deletes a live Python queue. Orphans from crashed Python processes are not swept automatically.
- Not in 0.1: waiting for MSMQ acknowledgments, multicast subscribers.

12.3 Testing

```
cd sdk\python
pip install -e .[test]
pytest                                    # codec golden vectors + client logic, any OS
# against a running server (verifies the COM transport):
$env:MQCACHE_TEST_ENDPOINT = "."; $env:MQCACHE_TEST_QUEUE = "mqcache"
pytest tests/test_integration_windows.py -v
```

`sdk/testdata/golden.json` is generated by the reference .NET codec (`dotnet run --project sdk/tools/GoldenVectors`). CI regenerates it and fails when it differs from the committed file, so the .NET codec and every SDK stay byte-compatible.

12.4 Writing another SDK

1. Implement chapter 8 (codec) and make it reproduce `sdk/testdata/golden.json`.
2. Implement the transport: send a message with `Label=mqcache.req`, `AppSpecific=1`, `BodyType=VT_ARRAY|VT_UI1 (0x2011)`, `ResponseQueue=<your private reply queue, DIRECT format name>`, then read the reply whose `CorrelationId` equals your message id.
3. Reuse the same request id on retries. Use your own queue-name prefix (not `mqcache.reply.` / `mqcache.sub.`).